Learning Management System (LMS) Analytics Using Hadoop Ecosystem

The Exponential Growth of LMS Data Requires Scalable Analytics



Unprecedented LMS Data Volume

Modern educational institutions generate massive volumes of learning data daily across diverse platforms.

Continuous Interaction Streams:

- Video views & engagement metrics
- Quiz attempts & automated scoring
- Discussion forum interactions & posts
- Document downloads & resource access

Every interaction feeds into continuous streams of clickstream data and unstructured activity logs.



The Necessity of Big Data Analytics

As digital learning adoption accelerates, traditional processing tools hit severe scalability bottlenecks.

Legacy Bottleneck:

Traditional architectures fail when querying terabytes of unstructured and semi-structured learning logs.

Essential Analytical Outcomes:

- Evaluating granular student engagement
- Identifying evolving learning patterns
- Measuring overall educational efficacy

Traditional Monolithic Databases Struggle with High-Volume Educational Logs

Analyzing high-volume learning activity logs presents significant challenges for traditional Relational Database Management Systems (RDBMS).



Scalability Bottlenecks

Vertical scaling becomes cost-prohibitive as historical log data grows exponentially.



Schema Rigidity

RDBMS requires strict schema-on-write, making it difficult to adapt to evolving, semi-structured LMS log formats.



Processing Latency

Executing complex aggregations and analytical queries over millions of transactional rows leads to severe performance degradation and timeouts in traditional monolithic architectures.

Distributed Storage and Processing Overcomes Educational Data Bottlenecks

To effectively process large-scale educational data, we must transition from centralized to distributed architectures. The **Hadoop ecosystem** provides the necessary foundation for this shift.



Horizontal Scalability

Distributed storage allows seamless expansion across commodity hardware clusters, eliminating compute capacity ceilings as institutional dataset sizes grow exponentially over time.



Fault Tolerance

Automated data replication ensures high availability of critical academic records, protecting sensitive institutional analytics against unexpected hardware or node-level failures.



Schema-on-Read Flexibility

Apache Hive enables SQL-like querying over unstructured logs without requiring heavy upfront data transformations, accelerating the time-to-insight for educational analysts.

Project Objectives

Decoding Student Behavior Through Big Data

This project aims to establish a robust big data pipeline to extract actionable educational intelligence. The core objectives include:

Student Engagement Analysis

Quantify interaction levels based on platform time and click rates.

Course Performance Evaluation

Assess overall academic success and completion metrics across different subjects.

Learning Behavior Profiling

Identify patterns in how students navigate course materials.

Risk Identification

Develop rule-based indicators to flag students exhibiting low-engagement behaviors, enabling proactive academic intervention.

Proposed Solution: An End-to-End Hadoop Analytics Pipeline

To achieve our objectives, we developed a comprehensive batch processing pipeline utilizing core Hadoop ecosystem components.

PHASE 01

Data Ingestion & Storage

Hadoop Distributed File System

Utilizing **HDFS** to reliably store massive volumes of raw LMS activity logs across distributed nodes.

```
Target Data: Raw LMS Logs
Pattern: Distributed Storage
```

PHASE 02

Analytical Processing

Apache Hive Infrastructure

Implementing **Apache Hive** as our data warehouse infrastructure to facilitate data summarization, ad-hoc querying, and analysis of large datasets stored in Hadoop files.

```
Engine: HiveQL / Data Warehouse
Capabilities: Ad-hoc Querying
```

PHASE 03

Batch Execution

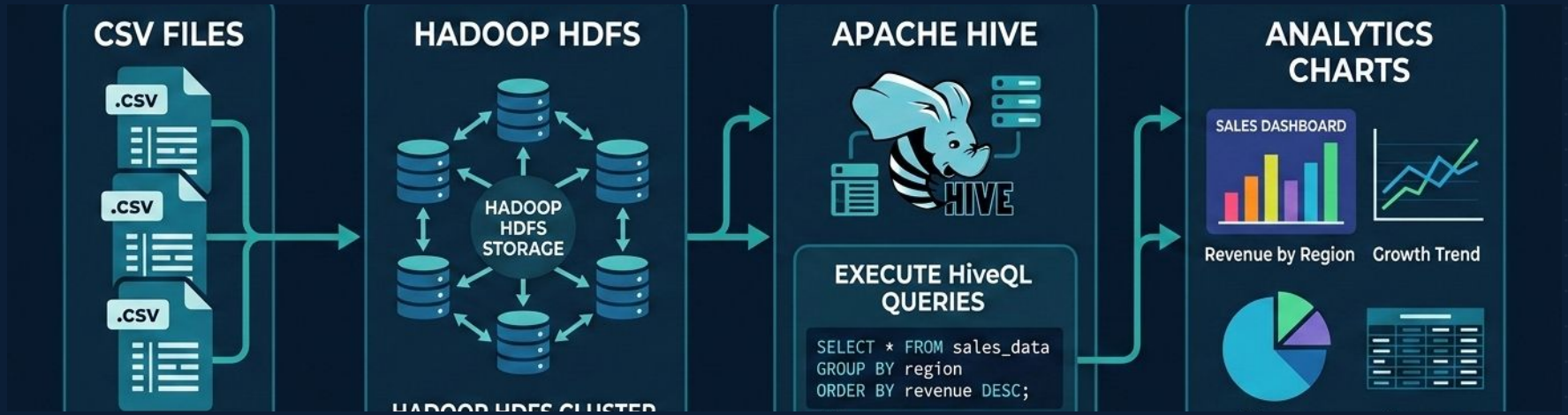
Automated Analytics Workflow

Designing an **automated workflow** for periodic batch analytics, transforming raw CSV logs into structured, queryable tables.

```
Input: Raw CSV Logs
Output: Structured Tables
```

ARCHITECTURE: Hadoop Distributed Cluster PIPELINE STATUS: Fully Functional Batch Model

Hadoop Ecosystem Architecture & Data Flow



1. Source

Raw LMS Activity

CSV files generated from LMS platform activity.

2. Ingestion

HDFS Loading

Loaded into directory path:

`/lms/input`

3. Warehouse

Hive Database

Database `lms` is initialized for metadata storage.

4. Tables

External & Managed

`activity_logs_10k`
`course_activity_10k`

5. Processing

HiveQL Execution

Aggregates performance metrics via MapReduce/Tez.

6. Output

Educational Insights

Actionable analytics and visualizations generated.

Dataset Specifications & Relational Data Model

10,000
Activity Records (Volume)

~1,000
Unique Students (Population)

3 Courses
Curriculum (C101, C102, C103)

ATTRIBUTE	DATA TYPE	DESCRIPTION / SEMANTICS
`student_id`	Integer / Key	Unique identifier for each enrolled student (~1,000 total cohort)
`course_id`	String / Key	Course code representing curriculum tracks (C101, C102, C103)
`session_id`	String / Key	Unique LMS interaction session identifier tracking active log-ins
`time_spent`	Float / Metric	Total active engagement time recorded per session (measured in seconds)
`clicks`	Integer / Metric	Frequency of interactive events and navigation actions per session
`completion_rate`	Float / Metric	Proportion of module content completed during session (percentage value)
`score`	Float / Outcome	Evaluated performance result tied to module assessments and activities

Data Modeling Value: This structured format enables deep cross-referencing between engagement metrics and academic outcomes.

HDFS Implementation: Establishing Distributed Storage

Data Migration & Cluster Ingestion Workflow

The initial phase involves moving data from local file systems into the distributed cluster. The step-by-step workflow includes:

1. **Dataset Prep**
   Formatting and cleaning the raw activity logs.
2. **HDFS Directory**
   Establishing the designated input path.
3. **Data Upload**
   Executing Hadoop put command for target path transfer.
4. **Verification**
   Utilizing HDFS list commands to ensure file integrity.
5. **Hive Readiness**
   Staging distributed data for schema mapping phase.

bash - hdfs@hadoop-cluster:~

```
# Step 2: Establish the designated input directory in HDFS
hdfs dfs -mkdir -p /lms/input

# Step 3: Execute Hadoop put command to transfer local logs to HDFS target path
hdfs dfs -put ./raw_data/activity_logs_10000.csv /lms/input/activity_logs_10000.csv

# Step 4: Verify block distribution, file size, and upload integrity
hdfs dfs -ls -h /lms/input/activity_logs_10000.csv
-rw-r--r-- 3 hdfs supergroup 2.4M 2026-10-05 18:09 /lms/input/activity_logs_10000.csv
```

Hive Implementation: Schema & Data Loading

Bridging Distributed HDFS Files to Structured Analytics Queries

Apache Hive bridges the gap between raw HDFS data and structured SQL execution through a 4-step sequence:

01 Database Initialization

Executes `CREATE DATABASE lms;` to isolate project tables and establish visual namespace boundaries.

02 Schema Definition

Creates base table `activity_logs_10k` mapping explicit data types directly to 7 core analytical attributes.

03 Data Mapping

Links Hive schema directly to external underlying HDFS directory path `/lms/input/`.

04 Data Cleansing

Applies table properties to explicitly skip header records during queries, preventing type-casting failures.

hive_schema_definition.hql

```
-- 1. Database Initialization
CREATE DATABASE IF NOT EXISTS lms;

USE lms;

-- 2. Base Schema Definition & Data Mapping
CREATE EXTERNAL TABLE IF NOT EXISTS activity_logs_10k (
log_id BIGINT,
user_id STRING,
course_id STRING,
action_type STRING,
duration_sec INT,
score_pct FLOAT,
timestamp_utc TIMESTAMP
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LOCATION '/lms/input/'

-- 3. Data Cleansing (Header Exclusion)
TBLPROPERTIES (
"skip.header.line.count"="1"
);
```

Optimizing Performance with Hive Partitioning

Implementation Mechanism

To maximize query efficiency, we implemented dynamic partitioning within Hive to segment massive activity datasets.

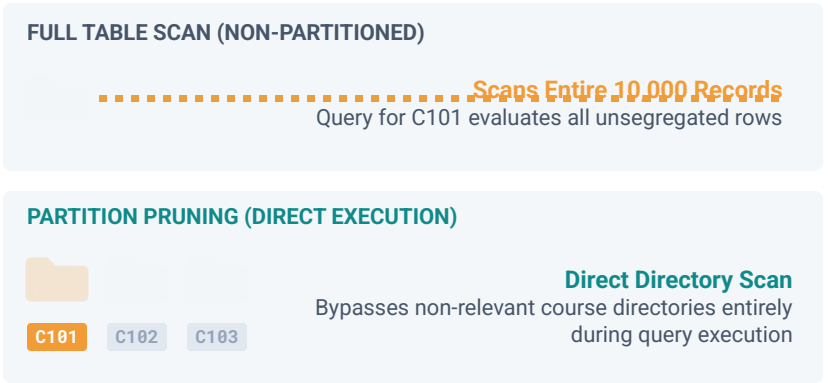
1. Explicit Partitioned Table

Created `course_activity_10k`, explicitly partitioned by `course_id` (C101, C102, C103).

2. Dynamic Partitioning Engine

Configured Hive to automatically distribute incoming logs into dedicated subdirectories during data load.

Directory Scanning Visualized



Partition C101

Isolated Course Segment

Direct access directory mapping

Partition C102

Isolated Course Segment

Automated dynamic data load

Partition C103

Isolated Course Segment

Targeted query optimization

Performance Benefits Summary

Partitioning drastically reduces data scanning. Hive scans only the targeted partition folder rather than the full 10,000-record dataset, yielding vastly improved query execution times.

Extracting Insights via HiveQL Analytics

With the data structured and partitioned, we executed targeted HiveQL queries to extract key educational metrics.

Record Verification

Standard count queries executed to validate the successful ingestion of 10,000 records.

Engagement Aggregations

Calculating average `time_spent` and total `clicks` grouped by `student_id` to build individual profiles.

Course-Level Statistics

Utilizing `GROUP BY course_id` to compute aggregate averages for `score` and `completion_rate` for comparative analysis.

hive-analytics-queries.hql

```
-- 1. Record Verification
SELECT COUNT(*) AS total_records
FROM student_logs_partitioned;

-- 2. Individual Engagement Profiles
SELECT student_id,
AVG(time_spent) AS avg_time,
SUM(clicks) AS total_clicks
FROM student_logs_partitioned
GROUP BY student_id;

-- 3. Course-Level Comparative Metrics
SELECT course_id,
AVG(score) AS avg_score,
AVG(completion_rate) AS avg_completion
FROM student_logs_partitioned
GROUP BY course_id;
```

Student Engagement Classification and Risk Analysis

We implemented a rule-based logic system in `HiveQL` to categorize student engagement and flag potential academic risks based on platform interaction metrics.

Highly Engaged

Tier 01 Cohort

High platform interaction frequency, consistent course progression, and active platform participation.

Moderately Engaged

Tier 02 Cohort

Regular platform usage meeting baseline requirements without demonstrating high-risk activity drops.

Low Engagement

Tier 03 Cohort

Sub-threshold interaction metrics triggering automatic academic risk flagging and interventions.

Risk Indicators & Threshold Rules

A student is flagged for **low-engagement risk** if they meet **any** of the following criteria:

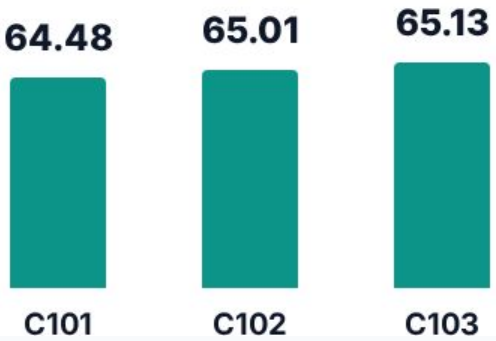
```
time < 500 OR clicks < 200 OR completion < 50%
```

Note: This represents a deterministic, rule-based demonstration of data filtering, serving as a foundational step prior to implementing complex Machine Learning models.

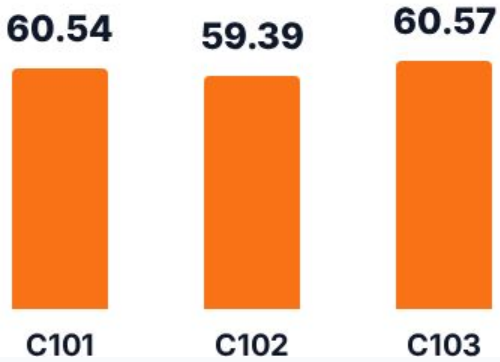
Course Performance Results

AGGREGATE ASSESSMENT INSIGHTS

Average Score



Average Completion (%)



Key Insights & Assessment Summary

Consistent Performance Baseline: Overall student scores remain tightly clustered across all courses (64.48 - 65.13).

Highest Achievement (C103): Web Development Basics leads both in average score (**65.13**) and completion rate (**60.57%**).

Completion Intervention: C102 shows a slight drop in completion (**59.39%**), marking it as a key focus area for engagement support.

Key Findings: Scalability and Educational Insights

The project successfully validated both the technical architecture and the educational data hypotheses.

Architectural Success

The Hadoop/Hive ecosystem demonstrated exceptional scalability, processing the 10,000-record dataset efficiently without traditional RDBMS bottlenecks.

Optimization Impact

Hive partitioning by `course_id` yielded measurable improvements in query execution speeds.

Behavioral Insights

The rule-based risk analysis successfully isolated distinct student cohorts, proving that fundamental engagement metrics (clicks and time) strongly correlate with final course performance and completion rates.

System Limitations and Future Scope

While successful, the current architecture presents opportunities for future enhancement.

Current Limitations

Synthetic Dataset Reliance

The evaluation currently relies on a synthetic dataset, which may not fully capture edge cases present in real-world student behavior.

Batch Processing Constraints

Operates strictly on fixed batch processing schedules, introducing latency in updating risk evaluations and student metrics.

Static Risk Rules

Uses static, hardcoded thresholds for risk analysis, limiting adaptability to evolving academic patterns across cohorts.

Future Scope



Kafka



Spark



ML Rules



Power BI

Real-Time Data Pipeline

Integrate **Apache Kafka** for real-time data ingestion and **Apache Spark** for low-latency processing to handle streaming behavioral data seamlessly.

Dynamic ML & Interactive Dashboards

Replace static rules with **Machine Learning** algorithms for dynamic dropout prediction. Connect the Hive warehouse to **Power BI** for interactive analytics.

Conclusion: Advancing Academic Analytics with Big Data

"This capstone project successfully demonstrates the practical application of the Hadoop ecosystem in solving real-world educational data challenges."

By migrating from monolithic databases to a distributed HDFS and Hive architecture, we established a scalable, fault-tolerant pipeline capable of analyzing complex learning behaviors.

The resulting insights into student engagement, course performance, and risk identification provide a strong foundation for data-driven decision-making and proactive academic interventions in higher education.

CAPSTONE SUMMARY STATEMENT • CSE-AIML ACADEMIC ANALYTICS

Thank You

Questions & Answers

STATUS: READY FOR FACULTY EVALUATION

Project Presentation Concluded.

